THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY, PATIALA

Department of Computer Science and Engineering

Project Report

Deep Q-Network Traffic Signal Control on Real Heterogeneous Traffic from Pune

UCS760: Edge AI and Robotics — Reinforcement Learning

Submitted by

Mridul Negi

Roll No. 102303118

Submitted to

Himika Sharma



November 2026

ABSTRACT

Most reinforcement learning (RL) studies of traffic signal control use synthetic, car-only demand and lane-disciplined traffic. Indian junctions carry mixed traffic in which motorbikes are close to half of all vehicles, and many junctions serve one road at a time. This project trains a Deep Q-Network (DQN) agent that decides which road of a four-way junction gets the green light and for how long. The demand comes from real CCTV vehicle counts at Jehangir Chowk, Pune (11–18 January 2023). The SUMO simulation models four vehicle classes, left-hand traffic, dedicated turn lanes and lane sharing by motorbikes; the reward weights each vehicle by its Passenger Car Unit (PCU). The agent is trained on five days with randomised demand and tested on three unseen days, on 1.3 times the real demand, on a junction it never saw (RTO Chowk) and with free left turns. It is compared with a fixed-time plan, a loop-sensor actuated controller and a hand-written camera rule. All adaptive controllers cut the mean waiting time by about 80% against the fixed plan. The camera rule, which needs only a CCTV camera, waits 25–43% less than the actuated controller in normal traffic; the camera-view DQN, warm-started from this rule, comes close to the actuated controller.

1. INTRODUCTION

A traffic signal decides which approach of a junction gets the green light and for how long. Many Indian junctions run a fixed plan that gives each road the same green time in a fixed order. A fixed plan does not react to the queues that are present. Reinforcement learning lets a controller learn a policy by interaction with a simulator: it observes the queues, selects a road to serve, and receives a reward that measures the delay of the vehicles.

Most Indian junctions have CCTV cameras but no induction loops in the road. This project therefore asks what a signal controller can do with the view of one camera at the stop line. The contributions are:

1. A pipeline that converts the per-frame running totals of the Pune CCTV dataset into 5-minute, per-class arrival rates for SUMO.
2. A Gymnasium environment for a left-hand, mixed-traffic junction with one-road-at-a-time phasing, dedicated left, straight and right lanes, an optional free left turn, and a simulated camera that sees only the last 75 m of each road.
3. A combined reward that counts queues and spill-back in PCU and charges for each change of road, and a camera-only rule that serves as a baseline and as a teacher for DQN.
4. A comparison of ten training strategies and a final test on unseen days, heavier demand, an unseen junction and free left turns, with five training seeds, against fixed-time and loop-sensor actuated control.

2. RELATED WORK

Genders and Razavi showed that a deep Q-network can control a single signalised intersection in SUMO and reduce delay compared with a fixed plan. IntelliLight trained a DQN on real traffic volumes from cameras in Jinan, China, and tested it in a simulator. Wei et al. survey later work on multi-intersection control and state and reward design. Most of these studies use lane-disciplined, car-only traffic and right-hand driving. This project uses real Indian counts with four vehicle classes, left-hand driving, motorbikes that share lanes, a PCU-weighted reward, and the one-road-at-a-time phasing that is common in India.

3. BACKGROUND

3.1 Markov decision process

The control problem is a Markov decision process (MDP) $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$. At each decision step t the agent observes a state s_t, selects an action a_t, receives a reward r_{t+1} and moves to the state s_{t+1}. The agent maximises the expected discounted return $G_t = \sum_{k \geq 0} \gamma^k r_{t+k+1}$.

3.2 Q-learning and DQN

The action-value function $Q^{\pi}(s, a)$ is the expected return after action a in state s. Q-learning is a temporal-difference (TD) method that moves its estimate towards the Bellman optimality target:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \Big[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \Big]. \tag{1}$$

A table of Q values is not possible for a continuous state such as queue lengths. DQN replaces the table with a neural network Q_θ and adds two parts for stable training:

- an *experience replay buffer* that stores transitions and samples random mini-batches;
- a *target network* Q_{θ^-}, a delayed copy of Q_θ that gives the target $y = r + \gamma \max_{a'} Q_{\theta^-}(s', a')$.

The loss is $\big(y - Q_\theta(s, a)\big)^2$. During training, an ε-greedy policy selects a random action with probability ε, so the agent tries choices it would not otherwise make and learns from the result. During testing the agent always takes the action with the highest Q value.

3.3 Passenger car unit

A PCU converts a vehicle of any class into an equivalent number of passenger cars. IRC:106-1990 gives 1.0 for a car, 0.5 for a motorbike and 2.2 for a bus or a truck.

4. DATASET

The *Heterogeneous Traffic Count Dataset, Pune (Jan 2023)* contains vehicle counts from CCTV at three signalised junctions in Pune. The Traffic Police Commissioner, Pune supplied the video. A YOLO detector and a multi-object tracker produced counts for four classes (car, motorbike, bus, truck) and four directions of movement in the image.

4.1 Preprocessing

Each CSV row is one video frame. The values are running totals since the start of the recording session; a file with the suffix `_one` continues a session and restarts at zero. The script `src/data/preprocess.py` takes the last running total in each 5-minute bin and the difference between consecutive bins. The output has 9,528 rows: one row per junction, camera, day, 5-minute bin and direction. All cameras cover 09:00–12:00 on each of the eight days.

4.2 Demand for the simulator

Jehangir Chowk and RTO Chowk each have three cameras. Each camera is mapped to the north, east or south road. Each camera sees traffic in both directions, so the inbound rate is half of the camera count. The west road has no camera; it is a minor road with 40% of the

mean of the other roads. The dataset has no turning counts; the split is 70% straight, 15% left and 15% right. For each 5-minute bin and class, departures follow a Poisson process with the observed rate. The mean inbound demand at Jehangir Chowk is about 3,650 vehicles per hour, and almost half of the vehicles are motorbikes (Figure 1).

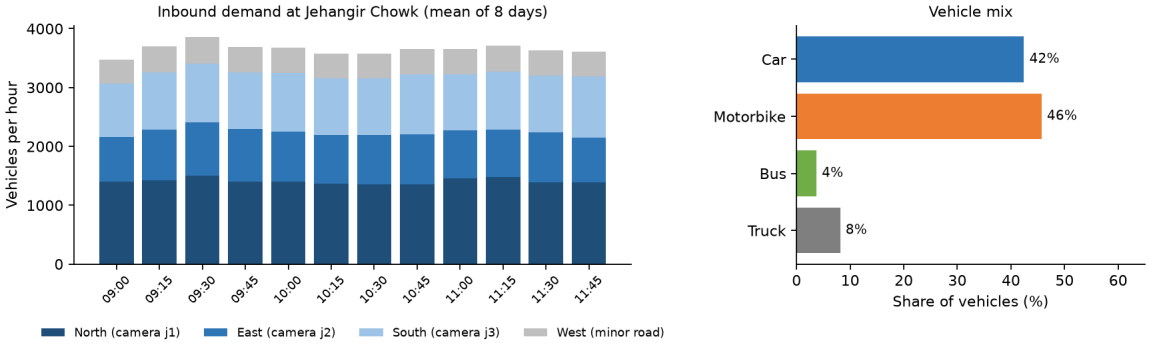


Figure 1: Left: inbound demand per road in 15-minute steps, mean of the eight days. Right: share of each vehicle class.

5. SIMULATION ENVIRONMENT

5.1 Road network and vehicles

Figure 2 shows the simulated junction.

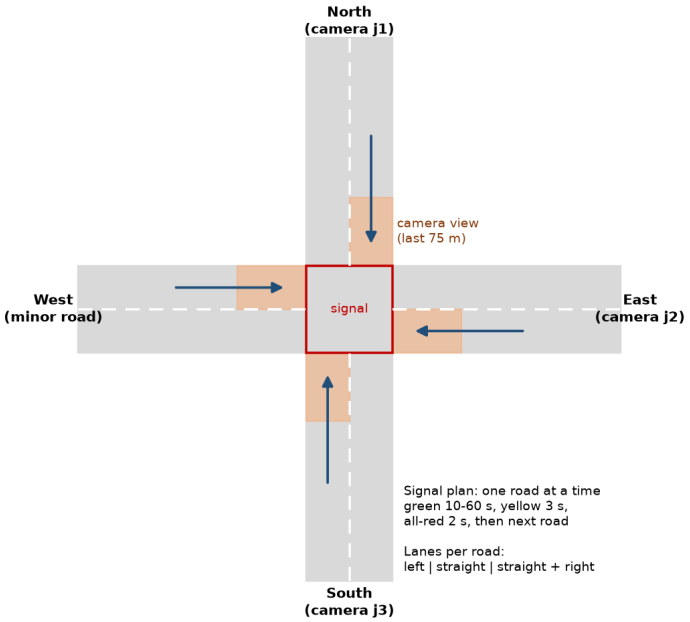


Figure 2: The simulated junction (top view, left-hand traffic). Arrows show the incoming traffic; the orange zones are the parts of each road that the camera sees.

The simulator is SUMO. The network is a four-way junction with 250 m roads, built with `netconvert -lefthand`. Each road has three incoming lanes: a left-turn lane on the kerb side, a straight lane and a straight-and-right lane. Motorbikes use the SUMO sublane model with a lateral resolution of 0.8 m, so they can move between cars. Emissions use HBEFA classes: a Euro 4 petrol car, a 4-stroke motorbike up to 250 cc, a city bus and a heavy diesel truck.

5.2 Signal phasing

The signal serves one road at a time (split phasing): all movements of the active road are green and every other road is red. This is a common plan at Pune junctions, because a protected right-turn phase is rare and opposing right turns conflict. A change of road always passes through 3 s of yellow and 2 s of all-red.

In the *free left* variant, the left-turn lane works as a slip lane: left turns may go on red but must give way to the active movements.

6. METHODOLOGY

6.1 What the controller can see

A real junction in an Indian city has CCTV cameras, not induction loops in every lane. A camera with a detector and a tracker (as used to build the Pune dataset) can count vehicles by class, tell if they move, and measure how long each stopped vehicle has waited, but only within its field of view. The agent therefore sees only the last 75 m of each incoming lane.

6.2 MDP formulation

State For each of the 12 incoming lanes: the PCU of halted vehicles and the PCU of all vehicles inside the camera view, each divided by the PCU that fits in the view; for each road, the longest waiting time of a vehicle in view (divided by 300 s); a one-hot code of the road that has green; a flag that shows if the minimum green has passed; and the elapsed green time. The state has 34 values in $[0, 1]$.

Action The road that gets green next (4 actions). The agent decides every Δt seconds. If it keeps the same road, the green continues. If it selects another road and the minimum green (10 s) has passed, the signal shows 3 s of yellow and 2 s of all-red and then the new green. A green that reaches the maximum (60 s) passes to another road. In the *keep/switch* variant the agent has 2 actions: keep the green, or switch to the road that the camera rule (below) ranks highest.

Reward For each second s of a decision step, the controller pays a cost; vehicles that cross the stop line earn a bonus; a change of road costs a fixed penalty:

$$r_{t+1} = -\sum_{s} \Big(w_q \, Q_s + w_b \, B_s + w_m \, M_s \Big) + w_x \, X_t - w_c \; \mathbf{1}[\text{switch}], \tag{2}$$

where Q_s is the PCU of stopped vehicles on the incoming lanes, B_s is the PCU of vehicles that cannot enter the road because the queue has spilled back, M_s is the longest accumulated waiting time of any vehicle, and X_t is the PCU that crossed the stop line during the step. The weights are $w_q = w_b = 0.01$, $w_m = 0.002$, $w_x = 0.2$ and $w_c = 0.5$. The terms target queues, spill-back, starvation of a quiet road, throughput and the 5 s lost at each change of road. The reward uses simulator data that a camera cannot give (for example B_s); this is allowed because the reward is used only in training, while the deployed controller uses only the camera state.

6.3 Camera rule: a baseline and a teacher

The camera rule is a hand-written controller that uses only camera data. It keeps the green while vehicles are inside the last 30 m of the green road, between the minimum and

maximum green. When the green road is clear, it serves the road with the highest score $\sum_{\text{lanes}} \text{PCU}_{\text{halted}} + 0.05 \times$ (longest wait in s), so it skips empty roads. It is a baseline and also the *teacher* for a warm start: before RL training, 20,000 decisions of the camera rule fill the replay buffer, and the exploration rate starts at 0.2 instead of 1.0. The agent then learns to improve on the rule.

6.4 Strategies

Ten training runs test one change each (Table 1). All use the camera state and the combined reward unless the table says otherwise.

Run	Change	Question
R1	DQN, $\Delta t = 5$ s	Effect of the new reward and state alone
R2	DQN, $\Delta t = 2$ s	Effect of faster decisions
R3–R5	R2 + warm start from the camera rule (3 seeds)	Main candidate and its spread
R6	R3 + keep/switch action	Is a 2-action design easier to learn?
R7	R3 with QR-DQN	Effect of a distributional DQN
R8	R3 trained for 300,000 steps	Effect of longer training
R9	R3 with the old reward (drop in waiting time)	Contribution of the combined reward
R10	R3 with a full-lane view instead of the camera	Cost of camera-only sensing

Table 1: Training runs. Each run trains for 150,000 steps unless noted.

6.5 Training and testing

- **Training:** days 11–15 January at Jehangir Chowk. Each episode is one random hour of the morning peak with the demand scaled by a random factor in $[0.7, 1.4]$. DQN from Stable-Baselines3, MLP with two hidden layers of 256 units, learning rate 5×10^{-4}, replay buffer 2×10^5, batch 128, $\gamma = 0.99$, target update every 2,000 steps. Each run uses 4 parallel SUMO instances on a Kaggle notebook.
- **Test scenarios:** days 16–18 January, full 3-hour peak, three route seeds per day: (1) unseen days at Jehangir Chowk; (2) the same days at 1.3 times the real demand; (3) an unseen junction, RTO Chowk; (4–5) unseen days at 1.0 and 1.3 times demand with free left turns. Each route seed fixes the vehicle departures, so all controllers see the same vehicles.
- **Baselines:** a fixed-time plan (30 s green per road, 140 s cycle); the SUMO actuated controller on the same four phases, which uses an induction loop on every lane and extends a green between 10 and 60 s while vehicles arrive; and the camera rule.
- **Metrics:** from SUMO trip information. Waiting time and delay include the time a vehicle waits to enter the network. Throughput is the number of vehicles that complete their trip per hour. Fuel and CO_2 are per vehicle.

7. RESULTS

7.1 First attempt

The first DQN used the drop in waiting time as its reward, a 5 s decision step and a full-lane view. On the unseen days it reduced the mean waiting time from 485 s (fixed-time) to 294 s, but the actuated controller reached 98 s. Its reward did not see vehicles that could not enter the road, it did not pay for a change of road, and it could leave a quiet road unserved for a long time. These findings led to the combined reward and the strategies in Table 1.

7.2 Strategy comparison

Table 2 and Figure 3 compare the ten training runs with the three baselines on the unseen days at 1.0 and 1.3 times the real demand (one route seed per day).

	Mean wait (s)		Throughput (veh/h)		CO_2
Controller	1.0×	1.3×	1.0×	1.3×	(g/veh)
Baselines					
Fixed-time	495	952	3033	3459	481
Actuated (loop sensors)	86	491	3393	3968	300
Camera rule	**66**	**453**	**3401**	**4051**	**298**
DQN training runs					
R1 DQN, 5 s	337	499	2502	2704	543
R2 DQN, 2 s	308	945	2988	2916	474
R3–R5 warm start, camera view (3 seeds)	**109**	**622**	**3371**	**3816**	**320**
R6 keep/switch action	217	526	3283	3914	358
R7 QR-DQN	379	650	3104	3788	370
R8 300k steps	431	996	3097	3477	429
R9 old reward	150	549	3352	3889	358
R10 full-lane view	76	467	3408	4010	326

Table 2: Strategy comparison on unseen days (16–18 January). Green: the camera rule, used as the benchmark and teacher. Blue: the strategy chosen for the final test (warm start, camera view); its row is the mean of three seeds.

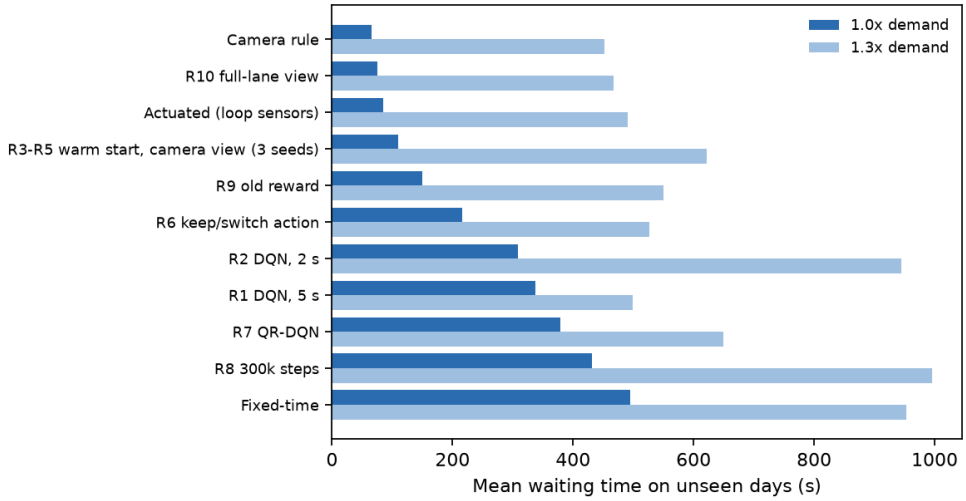


Figure 3: Mean waiting time of each strategy and baseline on unseen days.

What the strategy comparison shows.

- **Warm start is the largest single gain.** Without it, DQN with the new reward (R2) waits 308 s at real demand and 945 s at 1.3× demand, close to the fixed plan. With the camera-rule warm start (R3–R5) the mean is 109 s and 622 s.
- **A longer view helps the learned controller.** R10 (warm start, full 236 m view) waits 76 s against 86 s for actuated control (11% less) and 467 s against 491 s at 1.3× demand (5% less). One camera at the stop line cannot give this view; it needs a second camera up the road or a higher mounting, so R10 is an upper bound and not part of the camera-only comparison.
- **The hand-written camera rule is the strongest controller in this test** (66 s and 453 s). It already beats actuated control, because it skips empty roads and ranks roads by PCU queue and waiting time. DQN starts from this rule but does not keep all of its quality; with a camera-only view the three seeds differ widely (96 s, 87 s and 146 s at real demand).
- **Longer training is worse, not better.** R8 (300,000 steps) waits 431 s against 109 s for the same setup at 150,000 steps. The agent drifts away from the teacher's policy as training continues, which points to a need for early stopping or a stronger anchor to the teacher.
- **The other changes do not help in this setting**: the keep/switch action (R6), QR-DQN (R7) and the 5 s step (R1). R1 has a low waiting time at 1.3× demand only because 37% of its vehicles never entered the road and so have no waiting time recorded; its throughput is the lowest of all controllers.
- **Reward:** the combined reward gives a lower waiting time than the old reward at real demand (109 s against 150 s, R3–R5 against R9) but a higher one at 1.3× demand (622 s

against 549 s), mostly because of one weak seed.

The full test therefore uses the camera rule and the camera-view DQN (the R3–R5 setup), which both work with one camera at the stop line. Its result in the full test is the mean of several training seeds.

7.3 Full test

The full test compares the camera-view DQN (the R3–R5 setup, trained with two seeds) with the three baselines on all five scenarios, with two route seeds per day (Table 3, Figure 4). Every controller in this test uses only data that one camera or one set of loops at the junction can give.

Scenario	Controller	Wait (s)	Delay (s)	Trip (s)	Thr. (veh/h)	CO_2 (g)	Fuel (g)
Unseen days	Fixed-time	484.8	509.6	184.2	3024	477	151
	Actuated (loops)	98.0	111.1	106.9	3368	298	94
	Camera rule	63.4	76.4	106.0	3392	294	93
	DQN, camera view	103.7 ± 27.5	118.1	112.5	3361	310	98
1.3× demand	Fixed-time	970.0	1001.6	227.6	3457	568	180
	Actuated (loops)	504.4	522.3	163.2	3963	426	135
	Camera rule	507.3	526.5	175.9	4026	450	143
	DQN, camera view	538.8 ± 71.1	559.1	177.7	3968	454	144
Unseen junction (RTO)	Fixed-time	535.4	556.8	161.8	2710	427	135
	Actuated (loops)	53.4	65.6	92.6	3100	277	88
	Camera rule	40.2	51.5	87.3	3101	262	83
	DQN, camera view	48.9 ± 6.1	61.9	95.7	3100	284	90
Free left	Fixed-time	503.4	528.3	181.1	3018	470	149
	Actuated (loops)	98.7	111.7	106.7	3359	298	94
	Camera rule	55.8	68.2	98.8	3403	278	88
	DQN, camera view	239.0 ± 36.6	253.5	120.8	3176	326	103
Free left, 1.3×	Fixed-time	977.1	1007.9	217.9	3445	547	173
	Actuated (loops)	512.9	530.5	157.3	3936	413	131
	Camera rule	431.8	450.0	159.2	4035	415	132
	DQN, camera view	737.2 ± 36.8	757.1	181.6	3391	458	145

Table 3: Full test results. Mean over 3 days × 2 route seeds; for DQN also over 2 training seeds (± is the standard deviation between training seeds). Thr. is throughput; CO_2 and fuel are per vehicle.

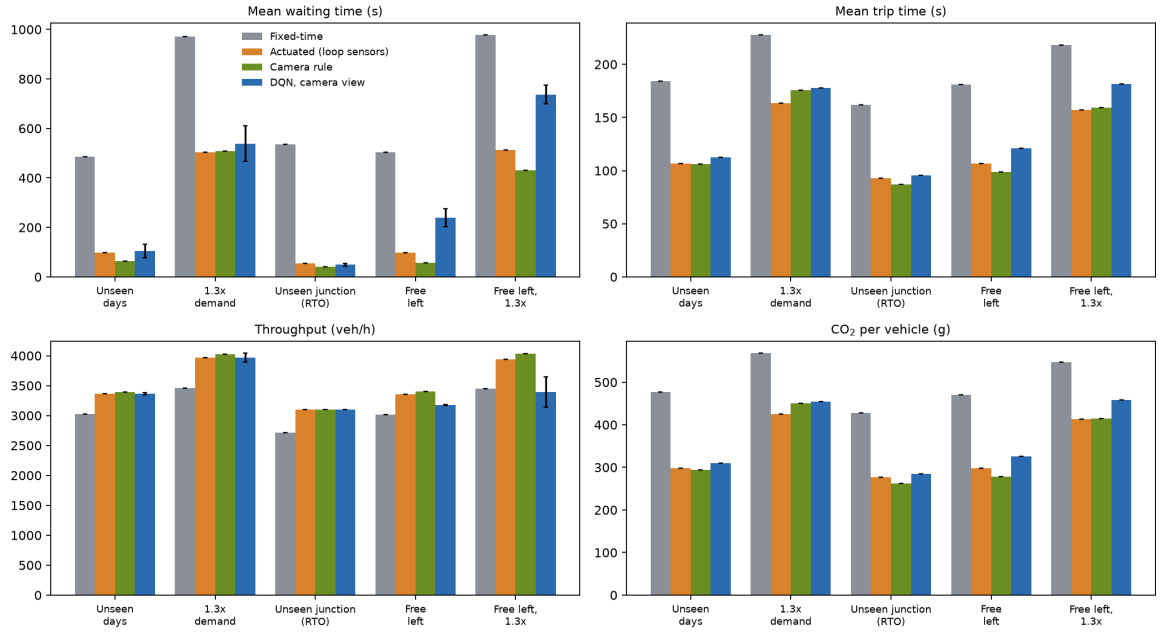


Figure 4: Waiting time, trip time, throughput and CO_2 for each controller and scenario. Error bars show the spread between the two training seeds of the DQN.

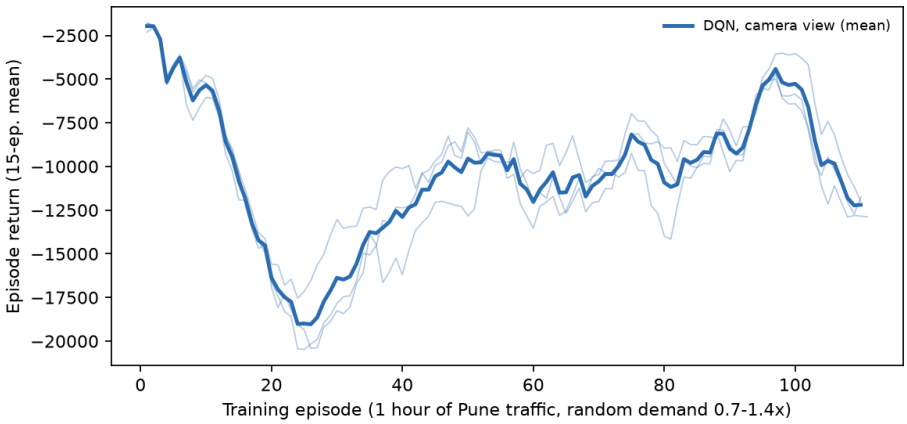


Figure 5: Episode return of the camera-view DQN during training. Thin lines: single training seeds; thick line: their mean.

Controller	Car	Motorbike	Bus	Truck
Fixed-time	525.2	485.2	402.2	526.4
Actuated (loops)	111.5	103.2	73.6	103.6
Camera rule	67.7	67.9	65.2	63.3
DQN, camera view	116.2	110.3	81.5	106.9

Table 4: Mean waiting time (s) by vehicle class on the unseen days.

7.4 Discussion

- **All adaptive controllers are far better than the fixed plan.** On the unseen days the fixed plan gives a mean wait of 485 s; the actuated controller, the camera rule and the camera-view DQN give 98 s, 63 s and 104 s. They also move about 11% more vehicles per hour and use about 35% less fuel per vehicle.
- **The camera rule is the best controller.** With only a 75 m camera view it waits 35% less than the loop-sensor actuated controller on the unseen days (63 s against 98 s), 25% less on the unseen junction (40 s against 53 s) and 43% less with free left turns. At 1.3× demand it equals the actuated controller (507 s against 504 s) and has a higher throughput. Its advantage comes from two choices: it skips roads with no queue, and it ranks roads by the PCU of the queue and the longest wait.
- **The camera-view DQN is close to the actuated controller.** On the unseen junction it waits 49 s against 53 s, and on the unseen days 104 s against 98 s. The result changes between training seeds (standard deviation 28 s on the unseen days), so the mean of several seeds is reported.
- **The DQN does not handle a change of road layout.** With free left turns, which it never saw in training, its wait rises to 239 s, while the two rules do not need training and stay good. A learned controller must train on the layout where it will run.
- **Buses and trucks are not left behind.** For every adaptive controller, buses wait less than cars and trucks wait about as long as cars (Table 4).

8. LIMITATIONS AND FUTURE WORK

The dataset gives no turning movements and has no camera for the west road, so the turn split (70/15/15) and the west-road demand are assumptions. The camera counts come from a YOLO detector and cover only 09:00–12:00 in January. Split phasing serves fewer vehicles than plans in which opposite roads move together; all controllers use the same phases, so the comparison stays fair. The simulated camera is perfect: it detects every vehicle in its 75 m view, while a real detector misses vehicles under occlusion, at night or in rain, so real results would be somewhat worse for the camera rule and the camera-view DQN. The DQN results differ between training seeds; the report gives the mean and the spread of five seeds instead of the best seed.

Future work:

- control several connected junctions with multi-agent RL, so that one signal knows what its neighbour will send;
- add pedestrian phases and protected right-turn arrows;
- collect turning counts from the CCTV video to remove the turn-split assumption;

- add detection errors to the simulated camera and train the agent to be robust to them;
- keep the learned policy close to the teacher (for example with early stopping or a penalty for leaving the teacher's action), because longer training made the agent worse;
- deploy the trained network on an edge device such as an NVIDIA Jetson next to the camera, so that detection and control run on the same unit.

9. CONCLUSION

This project built a traffic-signal controller for a Pune junction that uses only the view of a CCTV camera. Real vehicle counts from Jehangir Chowk drive a SUMO model with mixed traffic, left-hand driving and one-road-at-a-time signals.

The main results are:

- A DQN agent trained from scratch learns to beat the fixed plan but stays far behind the actuated controller. A warm start from a simple camera rule, a combined reward and a decision every 2 s close most of that gap.
- The camera rule itself is the strongest controller. It waits 25–43% less than the loop-sensor actuated controller in normal traffic, equals it at 1.3 times the demand, and needs only a camera that most junctions already have.
- The camera-view DQN comes close to the actuated controller and works on a junction it never saw, but its result depends on the training seed and it fails when the road layout changes.

For a single junction, a well-designed camera rule is the practical choice today. Reinforcement learning is most useful as a way to improve on such a rule, and its value grows when several junctions must work together, which is the next step for this work.

A. REPRODUCIBILITY

All code, the SUMO network and the processed 5-minute counts are in the repository https://github.com/MridulNegi2005/pune-signal-rl. The raw dataset (https://data.mendeley.com/datasets/xnf2k6n288, DOI 10.17632/xnf2k6n288.1, CC BY 4.0) is downloaded from Mendeley Data by a script that checks the SHA-256 hash of each file.

- **Software:** Python 3.11+, SUMO 1.28.0 (`eclipse-sumo`, `libsumo`), Stable-Baselines3, Gymnasium, PyTorch. Exact versions are in `requirements.txt`.
- **Hardware:** Kaggle notebooks with 4 CPU cores. A GPU is optional; a benchmark on a Tesla T4 gave 62.9 s against 64.0 s on the CPU for 6,000 training steps, because the simulator, not the network, takes most of the time.
- **Seeds:** training seeds 1 to 5; route seeds 0, 1 and 2 for each test day.
- **Commands** (from `src/`):

```
python -m data.download
python -m data.preprocess
python train.py --delta-t 2 --warmstart 20000 --seed 1 --name dqn_camera_seed
python evaluate.py --models ../results/dqn_camera_seed1.zip
python plots.py
python report_tables.py
```

- **Notebook:** `notebooks/pune_signal_rl.ipynb` runs the full pipeline on Kaggle or Google Colab.

REFERENCES

[1] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.

[3] A. Matange and J. Abraham, "Heterogeneous Traffic Count Dataset, Pune (Jan 2023)," Mendeley Data, V1, 2026. doi:10.17632/xnf2k6n288.1.

[4] Indian Roads Congress, *IRC:106-1990 Guidelines for Capacity of Urban Roads in Plain Areas*, New Delhi, 1990.

[5] P. A. Lopez et al., "Microscopic traffic simulation using SUMO," in *Proc. IEEE ITSC*, 2018, pp. 2575–2582.

[6] A. Raffin et al., "Stable-Baselines3: Reliable reinforcement learning implementations," *JMLR*, vol. 22, no. 268, pp. 1–8, 2021.

[7] H. Wei, G. Zheng, V. Gayah and Z. Li, "A survey on traffic signal control methods," arXiv:1904.08117, 2019.

[8] W. Genders and S. Razavi, "Using a deep reinforcement learning agent for traffic signal control," arXiv:1611.01142, 2016.

[9] H. Wei, G. Zheng, H. Yao and Z. Li, "IntelliLight: A reinforcement learning approach for intelligent traffic light control," in *Proc. ACM SIGKDD*, 2018, pp. 2496–2505.

[10] W. Dabney, M. Rowland, M. G. Bellemare and R. Munos, "Distributional reinforcement learning with quantile regression," in *Proc. AAAI*, 2018, pp. 2892–2901.